710

Synopsis

Alone on a weather-beaten offshore oil rig in the North Sea, communications are down, and a damaged automation terminal on backup power is the only chance to restore functionality and return home. You're only an apprentice. Each repaired circuit unlocks recovered terminal logs that hint that the shutdown may not have been so accidental. Fictional fault codes on a 7 segment display turn to "710" on level completion. When rotated, 710 reads OIL.

MDA Framework

Mechanics: Place and remove logic gates in fixed sockets, inspect truth tables, toggle input states, verify cases, and diagnose incorrect outputs. Progressively harder levels require players to construct and reuse compound modules (e.g. XOR from AND, OR and NOT).
Dynamics: Experiment, debug, and combine earlier solutions to solve more advanced challenges, like opening a complex interlock which takes inputs from oil pumps, master switches, etc.
Aesthetics: Isolation and suspense are communicated through green terminal-style logs, damaged status panels, classic 7 segment (calculator-style) numbers, and storm ambiance. The puzzles themselves will have a classic hacker look to them. Digital computery sounds for interactions over the top of the wind, rain and waves will make up the soundscape.

Core Gameplay Loop and Intended Scope

Read work order / target truth table --> construct circuit --> run all input combinations --> inspect faults and refine --> receive 710 verification --> unlock next work order.

Planned Content: Around 7-10 escalating puzzles, culminating in multi-input logic and reusing modules made in earlier levels.
Stretch Goals: Player-editable wiring between gates, drag-and-drop modules, and memory/latch puzzles.

Technical Approach, Architecture and Sketches

C++ with SDL3 will handle the interactive 2D interface, mouse input and rendering. OOP classes separate the game/level flow, circuit graph, gate sockets, connections and module library. The evaluator will support basic gates and verified subcircuits. Levels, wiring and targets will load from text files via standard C++ file I/O. Invalid/missing connections will be handled safely. *See figure 1 & 2 in the appendix for wireframe and UML.*

External Libraries and Intended AI Use

Libraries: SDL3 and the C++ standard library only.
AI: ChatGPT for concept/planning, debugging, and asset generation. Use will be logged and prompt examples given.

Development roadmap

21 - 25 Oct: Evaluator, SDL3 UI and early puzzles. **25 - 30 Oct:** File-loaded levels, truth-table testing and reusable modules. **31 Oct - 5 Nov:** Complete 7 - 10 levels, playtest, refine, document and record video. **6 Nov:** CW1 submission.

APPENDIX

Figure 1: Class Diagram

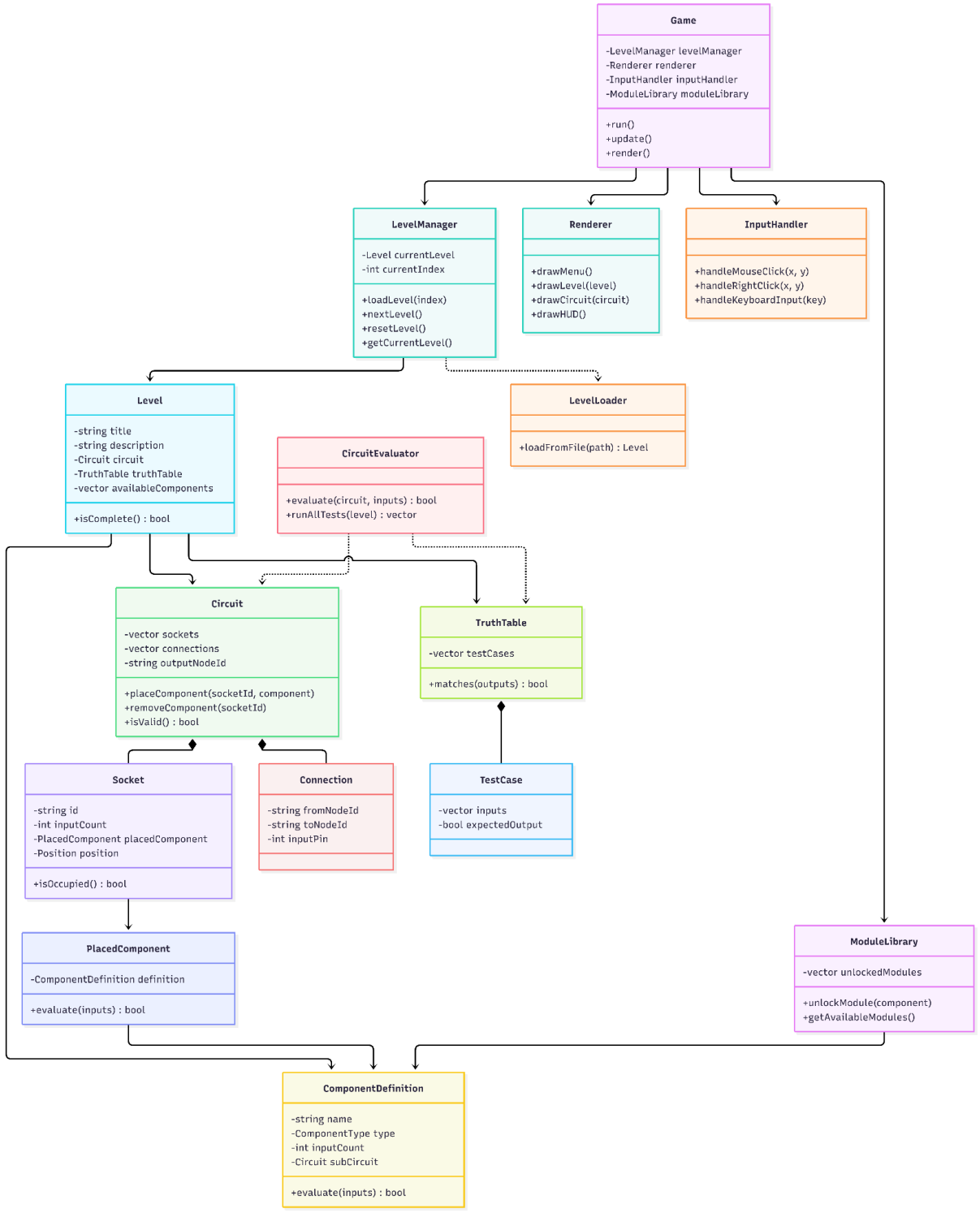


Figure 2: Main Puzzle UI Wireframe

